

---

# Lernzeit

## Der smarte Gruppenkalender

HKA - Software Labor Sommersemester 2026

Moritz Vogel

Simon Trinkl

# Inhaltsverzeichnis

|                                                                   |    |
|-------------------------------------------------------------------|----|
| 1. Grundlegende Problemstellung .....                             | 3  |
| 2. Architekturvorschlag .....                                     | 3  |
| 3. Technologieauswahl .....                                       | 5  |
| 4. Use Cases .....                                                | 6  |
| 5. Muss-/Kann-Kriterien .....                                     | 6  |
| 6. Umsetzung und Implementierung .....                            | 7  |
| 6.1. Frontend .....                                               | 7  |
| 6.1.1. Designprozess .....                                        | 7  |
| 6.1.2. Komponenten .....                                          | 9  |
| 6.1.3. API-Kommunikation und State-Management .....               | 11 |
| 6.2. Backend .....                                                | 11 |
| 6.2.1. Architekturprinzipien .....                                | 11 |
| 6.2.2. API-Endpunkte .....                                        | 12 |
| 6.2.3. Kalenderdaten-Verarbeitung .....                           | 13 |
| 6.2.4. Gruppenverwaltung .....                                    | 14 |
| 6.2.5. Datenbank .....                                            | 15 |
| 6.2.5.1. Datenbankstruktur .....                                  | 15 |
| 6.2.5.2. Implementierung .....                                    | 15 |
| 6.2.6. Authentifizierung .....                                    | 15 |
| 6.2.7. Kommunikation mit der RaumZeit API .....                   | 16 |
| 6.3. Testbarkeit .....                                            | 17 |
| 6.4. Deployment und Infrastruktur .....                           | 18 |
| 6.4.1. Deployment-Umgebung .....                                  | 18 |
| 6.4.2. Containerisierung und Orchestrierung mit .NET Aspire ..... | 18 |
| 7. Fazit .....                                                    | 19 |
| 7.1. Rückblick .....                                              | 19 |
| 7.2. Ausblick .....                                               | 20 |
| Literaturverzeichnis .....                                        | 21 |
| Hinweise auf KI-Unterstützung .....                               | 21 |

## 1. Grundlegende Problemstellung

Studierende stehen regelmäßig vor der Herausforderung, gemeinsame Termine für Lern- oder Projektgruppen zu koordinieren. Aufgrund individueller und häufig stark unterschiedlicher Stundenpläne gestaltet sich die Abstimmung geeigneter Zeitfenster als zeitaufwendig und ineffizient.

In der Praxis erfolgt die Terminfindung meist über Gruppenchats oder persönliche Absprachen. Dabei werden Vorschläge von einzelnen Gruppenmitgliedern eingebracht und anschließend von den übrigen Teilnehmenden auf mögliche Konflikte geprüft. Dieser iterative Abstimmungsprozess führt häufig zu einem „Hin und Her“-Austausch, der sowohl zeitintensiv als auch organisatorisch aufwendig ist.

Die Anwendung **Lernzeit** adressiert dieses Problem, indem sie die Terminfindung automatisiert unterstützt. Nutzerinnen und Nutzer können Gruppen erstellen und ihre individuell blockierten Zeiten hinterlegen. Auf Basis dieser Informationen generiert die Anwendung geeignete Terminvorschläge, zu denen alle Gruppenmitglieder verfügbar sind. Ziel ist es, den Abstimmungsaufwand zu reduzieren und eine effiziente sowie transparente Terminplanung zu ermöglichen.

## 2. Architekturvorschlag

Es wurden im Rahmen des Pflichtenhefts verschiedene Systemarchitekturen verglichen. Eine Gegenüberstellung hierzu ist in Tabelle 1 zu finden.

Es wurde sich im Rahmen des Projekts für eine Client-Server Architektur entschieden. Die erhöhte Komplexität alternativer Architekturstile steht in keinem angemessenen Verhältnis zu deren potenziellen Vorteilen für das Lernzeitprojekt im Rahmen des Labors.

Durch eine konsequente modulare Trennung der Funktionalitäten wird jedoch sichergestellt, dass eine spätere Migration zu einer Microservice-Architektur grundsätzlich möglich ist.

| Architektur                     | Pro                                                                                                                                                                                                          | Kontra                                                                                                                                                                                                          |
|---------------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Client-Server</b>            | <ul style="list-style-type: none"> <li>• Zentrale Kontrolle und Verwaltung</li> <li>• Einfache Implementierung</li> <li>• Gute Sicherheit durch Backend</li> <li>• Anonymisierung zentral möglich</li> </ul> | <ul style="list-style-type: none"> <li>• Single Point of Failure</li> <li>• Skalierungsprobleme bei hoher Last</li> <li>• Kann ggf. zu unwartbarem Monolithen wachsen</li> </ul>                                |
| <b>Microservice Architektur</b> | <ul style="list-style-type: none"> <li>• Gute Skalierbarkeit</li> <li>• Klare Trennung der Verantwortlichkeiten</li> <li>• Services unabhängig entwickelbar</li> <li>• Flexibel erweiterbar</li> </ul>       | <ul style="list-style-type: none"> <li>• Höhere Komplexität</li> <li>• Aufwendiges Deployment</li> <li>• Kommunikation zwischen Services notwendig</li> <li>• Fehler schwerer zu debuggen</li> </ul>            |
| <b>Peer-to-Peer</b>             | <ul style="list-style-type: none"> <li>• Keine zentrale Instanz nötig</li> <li>• Gute Skalierbarkeit</li> <li>• Geringe Serverlast</li> <li>• Hohe Ausfallsicherheit</li> </ul>                              | <ul style="list-style-type: none"> <li>• Komplexe Client-Logik</li> <li>• Sicherheits- und Datenschutzprobleme</li> <li>• Synchronisation schwierig</li> <li>• Abhängigkeit von Client-Verfügbarkeit</li> </ul> |

Tabelle 1: Vergleich System Architekturen

Als Protokoll zur Kommunikation wird eine REST-Schnittstelle verwendet. Es wurde als Alternative GraphQL betrachtet, allerdings scheint auch hier durch die höhere Komplexität bei der Implementierung im Rahmen des Projekts keinen Mehrwert zu bieten.

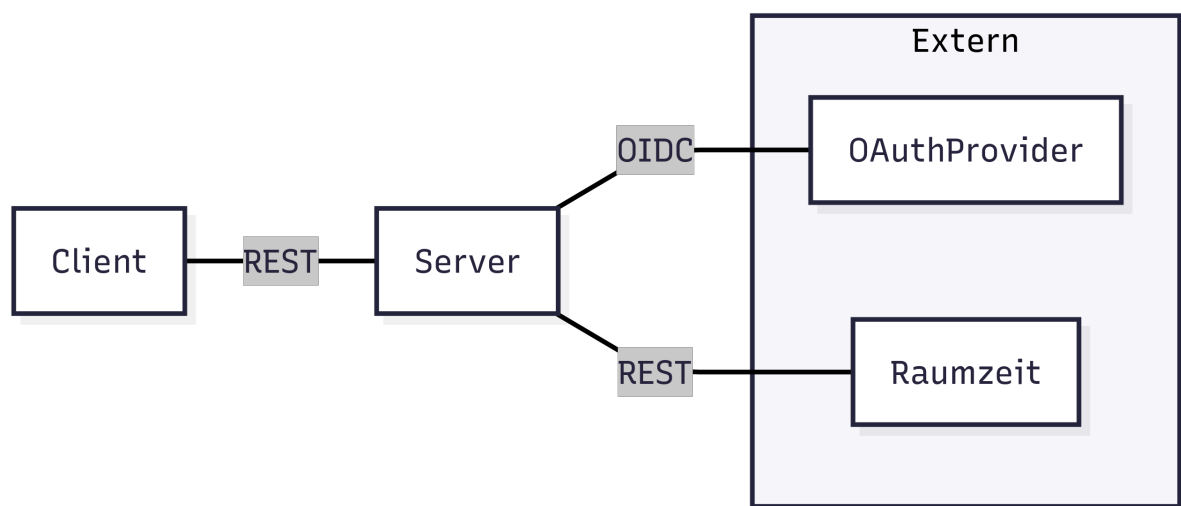


Abbildung 1: Systemarchitektur Lernzeit

### 3. Technologieauswahl

Im Rahmen der Analyse wurden für das Backend die Programmiersprachen *Rust*, *Go* und *C#* sowie für das Frontend die Frameworks *Blazor*, *Vue* und *React* evaluiert. Die Tabellen Tabelle 2 und Tabelle 3 listen die jeweiligen Stärken und Schwächen übersichtlich auf.

|      | Pro                                                                                                                                                                                                           | Kontra                                                                                                                                                                                                  |
|------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|---------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Rust | <ul style="list-style-type: none"> <li>• Höchste Performance</li> <li>• Memory-Safe (Ownership &amp; Borrowing)</li> </ul>                                                                                    | <ul style="list-style-type: none"> <li>• Noch keine Erfahrung im Team</li> <li>• Steile Lernkurve</li> <li>• Saubere Schichtentrennung komplex</li> <li>• Wenige SDKs / externe Bibliotheken</li> </ul> |
| Go   | <ul style="list-style-type: none"> <li>• Erfahrung im Team</li> <li>• Sehr schnelle Entwicklung &amp; einfache Syntax</li> <li>• Einfaches Deployment / kleine Container</li> <li>• Minimalistisch</li> </ul> | <ul style="list-style-type: none"> <li>• Fördert keine saubere Architektur (wenig Guidance)</li> <li>• DI nur durch externe Bibliothek</li> <li>• Weniger Features für komplexe Patterns</li> </ul>     |
| C#   | <ul style="list-style-type: none"> <li>• Erfahrung im Team</li> <li>• Ausgereiftes Ecosystem, viele SDKs</li> <li>• Built-in DI &amp; klare Layer-Struktur</li> </ul>                                         | <ul style="list-style-type: none"> <li>• Container größer, Startup langsamer</li> <li>• Framework kann „Magic“ verstecken</li> </ul>                                                                    |

Tabelle 2: Vergleich Backend Technologien

|        | Pro                                                                                                                                                                                                                  | Kontra                                                                                                                                                                                                                                                      |
|--------|----------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|-------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| Blazor | <ul style="list-style-type: none"> <li>• Fullstack mit C# Backend möglich</li> <li>• Starke Typisierung &amp; Compile-Time Checks</li> <li>• Gute Integration in .NET Ecosystem</li> </ul>                           | <ul style="list-style-type: none"> <li>• Wenig Erfahrung im Team</li> <li>• Weniger reifes Frontend-Ökosystem als JS Frameworks</li> <li>• Lernkurve für WebAssembly-spezifische Konzepte</li> <li>• Weniger Community-Beispiele &amp; Tutorials</li> </ul> |
| Vue    | <ul style="list-style-type: none"> <li>• Erfahrung im Team</li> <li>• Sehr einfache Lernkurve, leicht verständlich</li> <li>• Flexible Komponentenstruktur</li> <li>• Große Community &amp; viele Plugins</li> </ul> | <ul style="list-style-type: none"> <li>• Architektur nicht erzwungen</li> <li>• Bei großen Projekten kann State-Management komplex werden</li> <li>• TypeScript optional, sonst lose Typisierung</li> </ul>                                                 |
| React  | <ul style="list-style-type: none"> <li>• Erfahrung im Team</li> <li>• Riesiges Ecosystem &amp; Community</li> <li>• Flexibles Komponentenmodell &amp; Hooks</li> <li>• TypeScript-Integration möglich</li> </ul>     | <ul style="list-style-type: none"> <li>• Kein festes Architektur-Muster vorgegeben</li> <li>• Boilerplate bei komplexem State / Redux</li> <li>• Lernkurve bei Hooks &amp; Patterns für Anfänger</li> </ul>                                                 |

Tabelle 3: Vergleich Frontend Technologien

Nach Abwägung der Vor- und Nachteile fiel die Wahl auf die Kombination C# / ASP.NET als Backend-Framework und React als Frontend-Framework.

Diese Entscheidung basiert auf folgenden Kriterien:

- **Architektur:** C# ermöglicht klare Layer-Strukturen und eingebaute Dependency Injection, während React eine flexible Komponentenstruktur bietet, die eine saubere Trennung von Zuständen und Logik unterstützt.
- **Team-Expertise:** Beide Technologien sind im Team bekannt, was eine schnelle Entwicklung und Wissenstransfer erleichtert.
- **Ökosystem & Community:** React und C# bieten jeweils ein umfangreiches Ökosystem, was die Implementierung von komplexen Anforderungen erleichtert. Beide Technologien sind etabliert und haben große Communities.
- **Lern- und Weiterentwicklung:** Die Kombination erlaubt es dem Team, vorhandenes Wissen zu vertiefen und neue Best Practices im Bereich Softwarearchitektur gemeinsam zu erlernen.

## 4. Use Cases

**1. Stundenplan importieren:** Die App kann den eigenen Uni-Stundenplan übernehmen. Dafür meldet man sich auf der Hochschul-Plattform an, erstellt einen Freigabe-Link und fügt diesen in Lernzeit ein.

**2. Lerngruppe erstellen:** Um gemeinsame Termine zu planen, erstellt man eine neue Lerngruppe und gibt ihr einen aussagekräftigen Namen - zum Beispiel „Mathe-Lerngruppe“.

**3. Teilnehmer einladen:** Wer eine Gruppe erstellt hat, kann andere über einen Link oder QR-Code weitere Personen einladen. Der QR-Code dient zu einer schnellen und unkomplizierten Art einer Lerngruppe beizutreten.

**4. Freie Zeiten finden und buchen:** Die App zeigt Zeiten, in denen alle Gruppenmitglieder frei sind. Diese freien Zeiten kann man als gemeinsame Lernzeit auswählen und optional einen Treffpunkt sowie eine Uhrzeit festlegen.

## 5. Muss-/Kann-Kriterien

|                             |                                                                                                                                                                                                                                                                                                                                                        |
|-----------------------------|--------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------------|
| <b>Mindestanforderungen</b> | <ul style="list-style-type: none"><li>• Importieren des eigenen Stundenplanes</li><li>• Erstellen von Lerngruppe</li><li>• Hinzufügen von neuen Mitgliedern durch einen Einladungslink oder einen QR-Code</li><li>• Abgleich von allen Kalenderdaten der Gruppenmitglieder</li><li>• Visuell überschaubare Darstellung der freien Zeitblöcke</li></ul> |
| <b>Nice to have</b>         | <ul style="list-style-type: none"><li>• Möglichkeit zur Abstimmung eines Termins</li><li>• Anzeige aller frei verfügbarer Räume auf dem Campus</li><li>• Benachrichtigungen bei neuen Terminvorschlägen oder Änderungen</li><li>• Integration von Kalender-Apps (Google Calendar, Outlook)</li></ul>                                                   |

## 6. Umsetzung und Implementierung

### 6.1. Frontend

#### 6.1.1. Designprozess

Zu Beginn wird ein Prototyp in Figma erstellt. Dabei wird das in Figma integrierte KI-Tool genutzt. Dies ermöglicht das Erledigen von Routineaufgaben (Resizing, Layouts, Styles), ein schnelleres Iterieren mehrerer Ideen und Designvarianten sowie eine Unterstützung der Umsetzung von barrierefreiem und inklusivem Design. [1]

Jedoch wirkt das generierte Design anfangs wenig durchdacht und generisch.



Abbildung 2: Iniziale Seite von Figma AI

Die mit dem KI-Tool erarbeitete Oberflächenstruktur wird nun verfeinert. Dazu werden die Seiten skizziert und der User-Flow von Seite zu Seite organisiert.

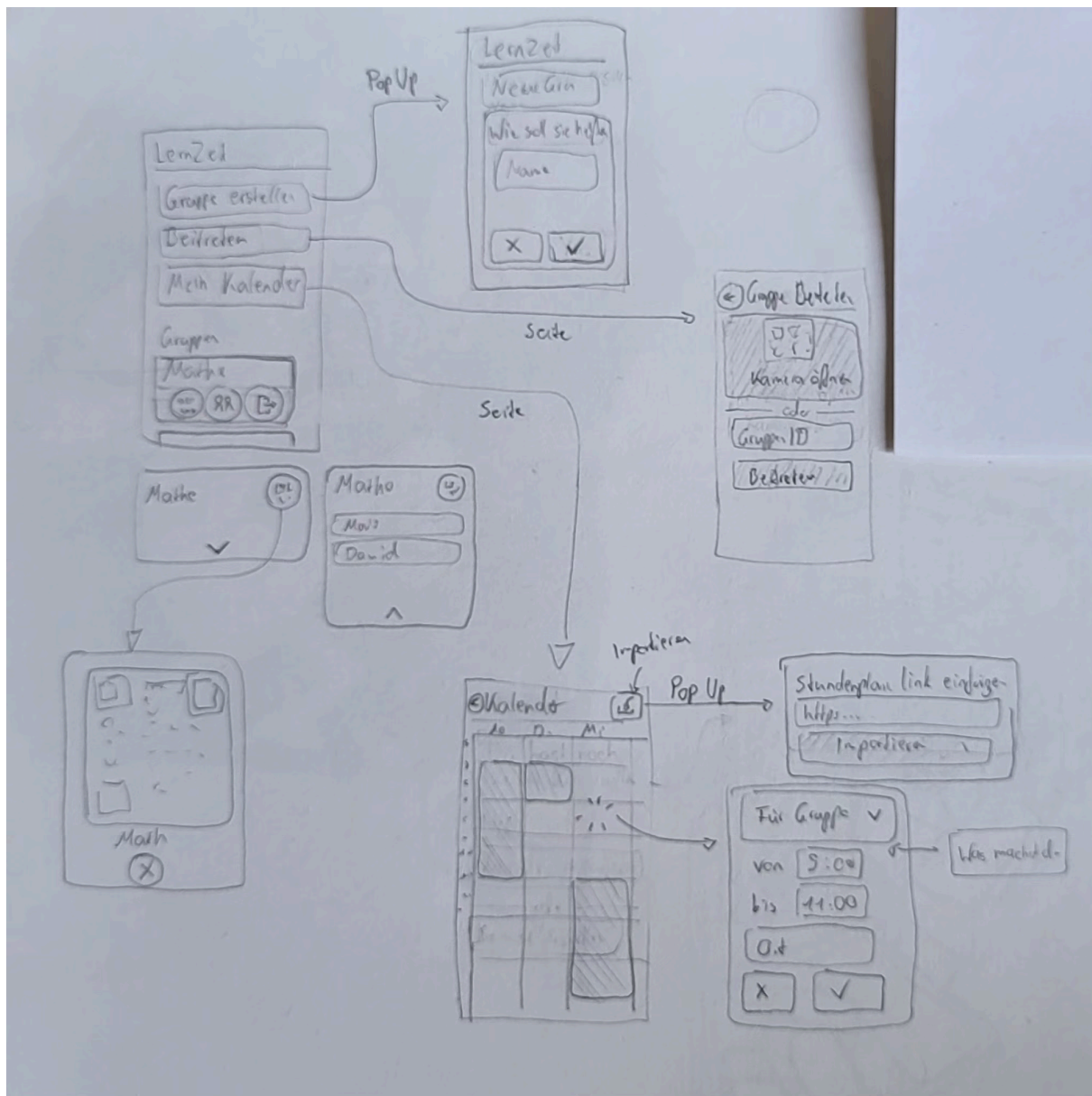


Abbildung 3: Skizze zum User-Flow

Um das visuelle Erscheinungsbild der Oberfläche festzulegen, wird ein Moodboard erstellt.

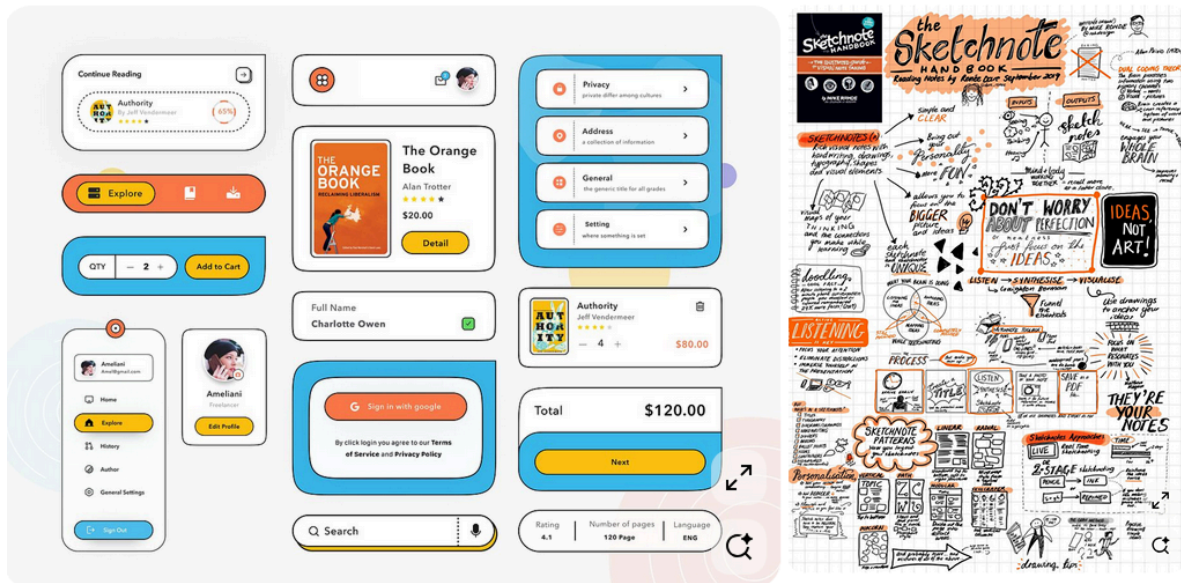


Abbildung 4: Das Moodboard [2], [3]

Das Moodboard hilft dabei, die visuellen Komponenten im Designprozess greifbar zu machen. Von den Referenzen ausgehend, werden die einzelnen Komponenten erstellt und in Figma zusammengeführt.

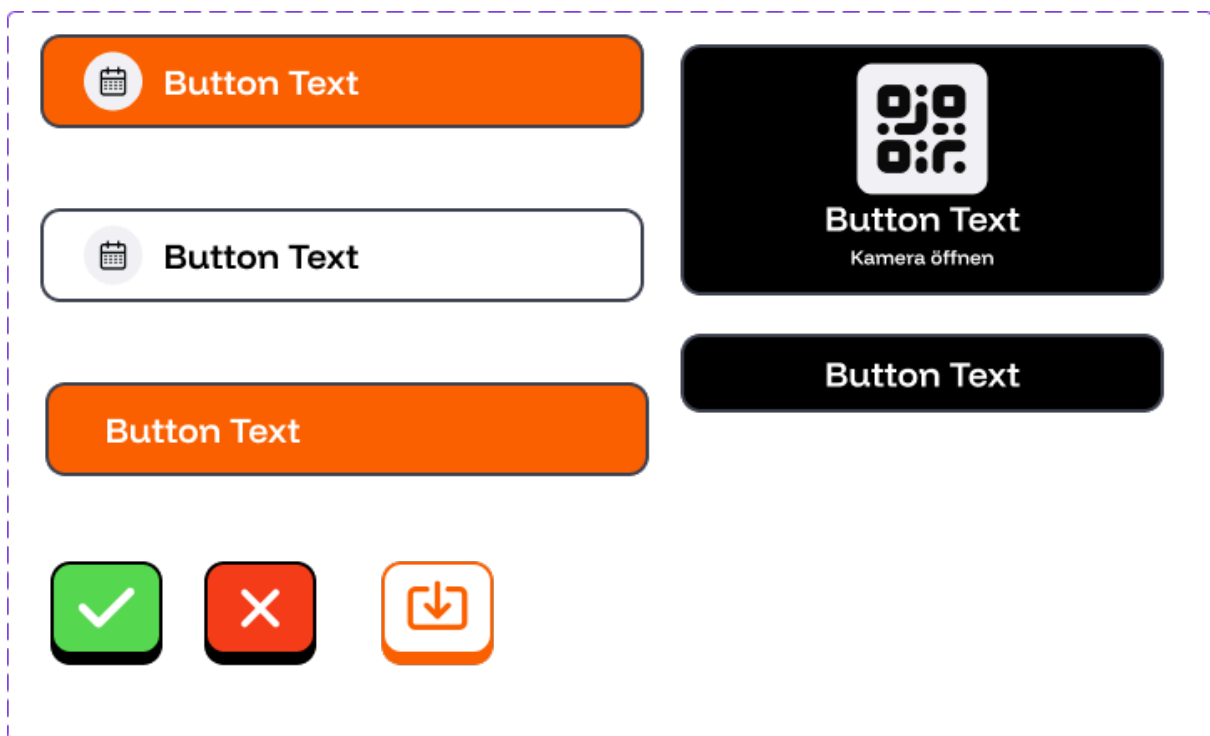


Abbildung 5: Iniziale Seite von Figma AI

### 6.1.2. Komponenten

Die Frontend-Architektur von **Lernzeit** folgt einem streng modularen Ansatz unter Verwendung von React. Das Ziel war es, eine einfach bedienbare Benutzeroberfläche zu schaffen, die es einfach ermöglicht Kalenderdaten verständlich zu visualisieren.

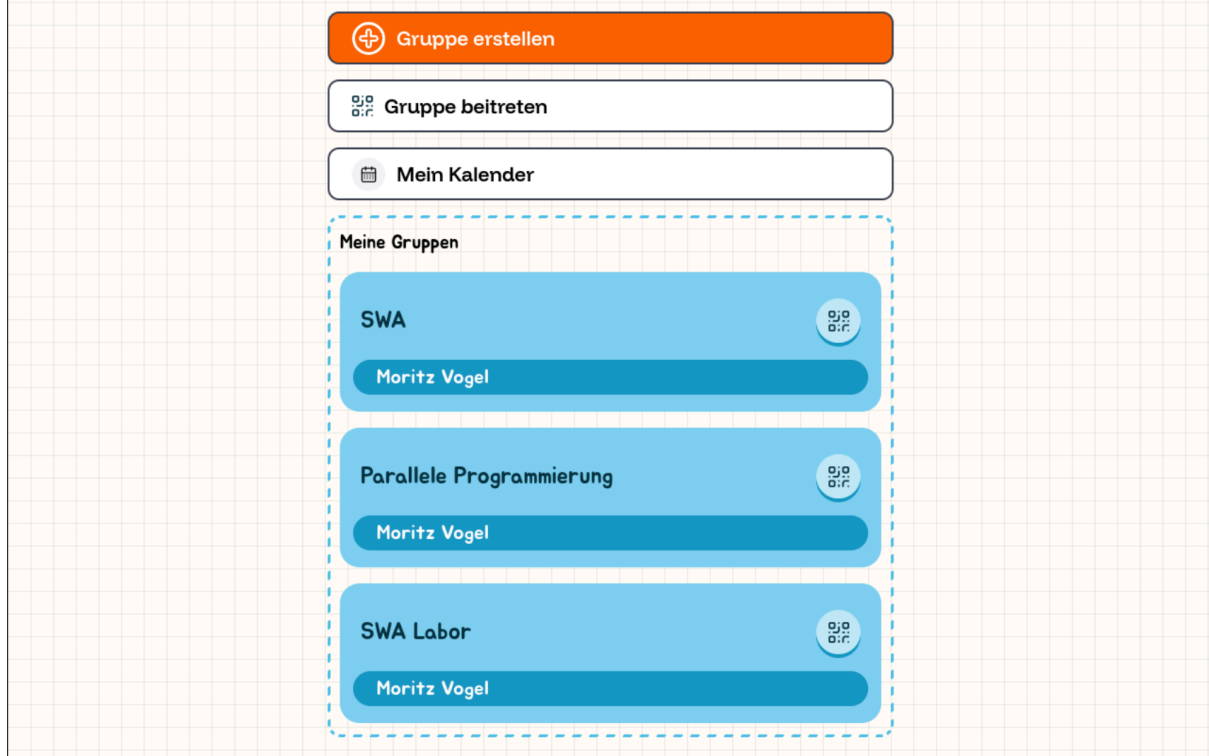


Abbildung 6: Übersicht der Hauptkomponenten in der Web-Oberfläche

Im Hinblick auf Softwarearchitektur wurde hier darauf geachtet, dass das UI möglichst modular gestaltet ist und Komponenten leicht wieder zu verwenden sind.

Hierbei sind folgende Komponenten entstanden:

- **Timetable-Komponente:** Dies ist das Herzstück der Anwendung. Sie visualisiert Zeitfenster in einem wöchentlichen Raster. Die Logik zur Berechnung der relativen Positionen der Event-Boxen ist in der Komponente gekapselt. Sie wird sowohl beim persönlichen, als auch beim Gruppenkalender verwendet um die Kalenderdaten anzuzeigen.
- **Button:** Die Button Komponente wird an vielen Stelle verwendet und bietet vielseitige Möglichkeiten um Symbole und Text zu integrieren und verschiedene Varianten des Buttons anzuzeigen.
- **GroupCard:** Ein wiederverwendbares Element zur Darstellung von Gruppeninformationen, das den schnellen Wechsel zwischen verschiedenen Lerngruppen ermöglicht. In Abbildung 6 sind die GroupCards zu erkennen.
- **Interaktive Modals:** In der Anwendung gibt es an vielen Stellen Modale, welche den Nutzer zu Aktionen auffordern. Zum Beitreten zu Gruppen, beim Hinzufügen eines Kalenders und beim Erstellen von Gruppen kommen diese zum Einsatz. Hierfür gibt es eine Basiskomponente, welche zum Erstellen der verschiedenen Modale verwendet wird.
- **Header & Navigation:** Eine konsistente Navigationsleiste, die den Nutzerstatus (Login/Logout via Google) reflektiert und schnellen Zugriff auf die Profil- und Gruppeneinstellungen bietet.

### 6.1.3. API-Kommunikation und State-Management

Die Anbindung an das Backend erfolgt über eine dedizierte Schicht im Frontend, die in `client.ts` definiert ist. Wir setzen hierbei auf die native `fetch-API`, ergänzt um `Error-Handling-Wrapper`.

In dieser Typescript Klasse werden alle Backend Endpunkte als Methoden definiert und können dann in React Hooks wie `useEffect` aufgerufen werden. Durch den Einsatz von DTOs im Backend ist hier eine klare Schnittstelle definiert, wodurch die Übertragung von den Daten auch bei Änderungen im Backend stabil bleiben kann.

## 6.2. Backend

Das Backend ist als ASP.NET Core Web API realisiert und folgt den Prinzipien der Clean Architecture (Trennung von Domain, Application und Infrastruktur).

### 6.2.1. Architekturprinzipien

Strukturell haben wir uns im Backend für eine hexagonale Architektur (auch bekannt als „Ports und Adapter“) entschieden. Hierbei werden spezifische Technologien in Adaptern abgekapselt und somit strikt von der Domänenlogik separiert. Die verschiedenen Schichten und Adapter werden in C# als separate Projekte modelliert. Hierdurch können Zugriffe der Schichten in falscher Richtung, also von innen nach außen, strukturell verhindert werden. Im folgenden Abschnitt werden die verschiedenen Schichten erläutert:

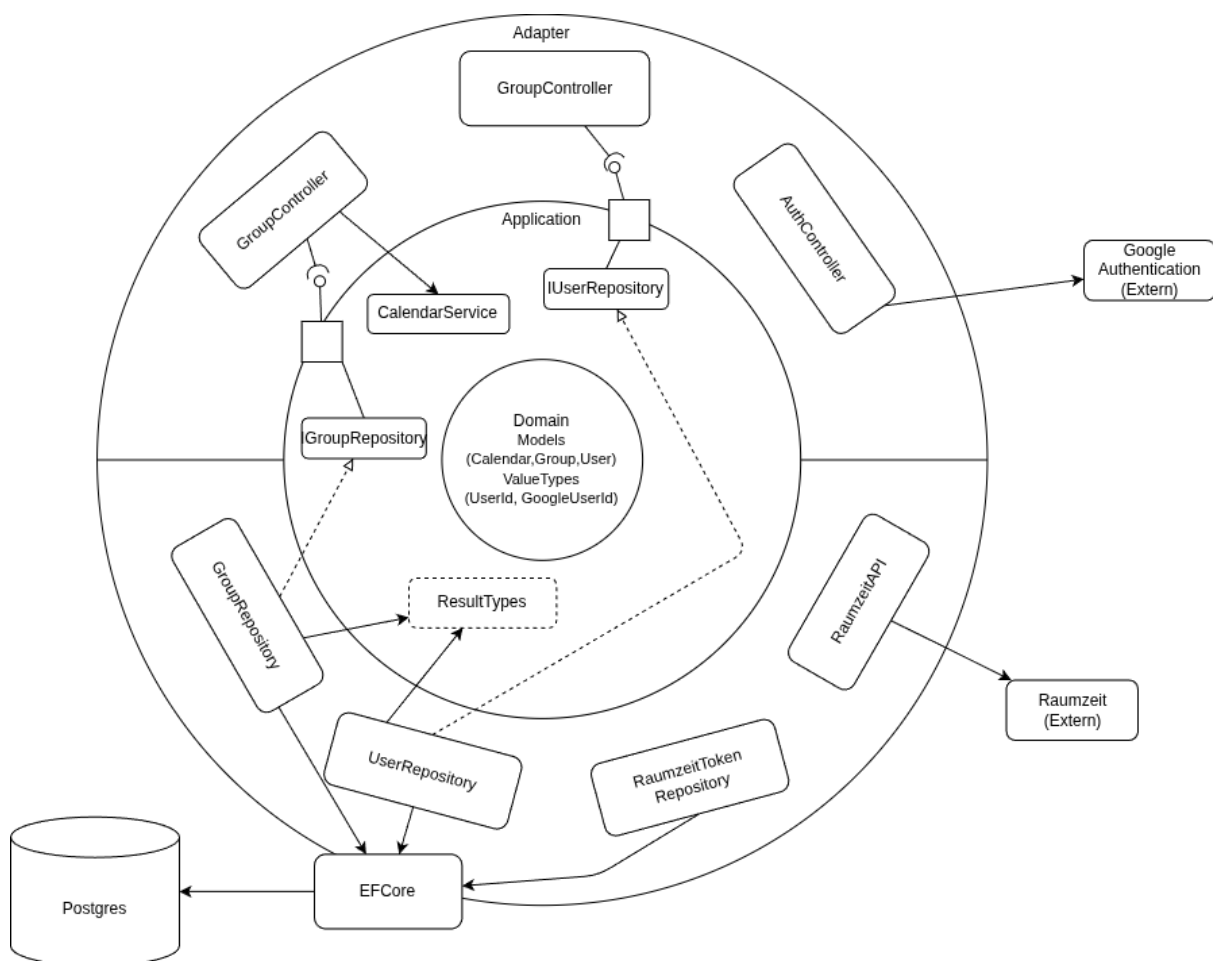


Abbildung 7: Architektur des Projekts

- **Domain:** Hier ist das Domänenmodell in Form von C#-Records definiert. Im C#-Kontext werden Records als Datenhüllen verwendet, da diese standardmäßig immutable (unveränderlich) sind. Dies hilft dabei, unerwartete Seiteneffekte in der Domäne zu vermeiden. Es wird empfohlen, in diesem Projekt den Prinzipien des Domain-Driven Designs (DDD) zu folgen und beispielsweise nach Möglichkeit Value Objects zu verwenden
- **Application:** In dieser Schicht erfolgt die Verknüpfung der Schichten. Hier werden die Schnittstellen (Ports) in Form von Interfaces definiert. Zudem werden dort Services implementiert, welche die Verarbeitung von Domänenobjekten und die Logik der Anwendungsfälle (Use Cases) definieren. In unserem Beispiel erfolgt hier die Berechnung der Gruppenkalender.
- **Adapter:**
  - **ASP.NET Core Webprojekt:** Hier wird die REST-Schnittstelle implementiert, über die Anwender:innen mit der Anwendung interagieren. In diesem Adapter werden Domänenobjekte zu DTOs (Data Transfer Objects) gemappt, um sie an das Frontend zu übergeben
  - **EFCore:** Entity Framework Core dient als ORM für die Interaktion mit der PostgreSQL-Datenbank. Hier werden Repositories implementiert, um Domänenobjekte zu speichern und zu laden. Die Domänenmodelle werden nicht direkt via EFCore persistiert, sondern zunächst auf separate Persistenz-Objekte gemappt, um keine technischen Details (wie Datenbank-Annotationen) in die Domäne propagieren zu lassen
  - **DataProtection:** Hier wird ein Dienst implementiert, der Tokens sicher verschlüsselt, damit keine unverschlüsselten Tokens in die Datenbank geschrieben werden.
  - **RaumzeitAPI:** Dieser Adapter realisiert die Anbindung an die externe Raumzeit-Schnittstelle via REST. Über den DataProtection-Adapter werden die Tokens sicher verwaltet. Geladene iCal-Kalender werden mit einer Library geparsed und in das interne Domänenmodell überführt.

### 6.2.2. API-Endpunkte

Um die Zuständigkeitsbereiche der Adapter klar zu gliedern, wurden diese in die drei Hauptbereiche Authentication, Groups und User unterteilt. Das Adaptermodell wurde wie folgt in die Endpunkte überführt:

#### Authentication[4]

Für die Authentifizierung der Nutzer fiel die Entscheidung auf einen Drittanbieter: in diesem Fall Google. Der Google-Authentifizierungsprozess bietet Nutzern eine komfortable Möglichkeit, sich mit nur einem Schritt bei Apps oder Websites anzumelden, indem sie ihren bestehenden Google-Account verwenden. Damit entfällt die Notwendigkeit separater Zugangsdaten, komplexer Registrierungsformulare und vergessener Passwörter. Mit nur wenigen Codezeilen lässt sich der Login-Prozess integrieren. Weitere Details zum Authentifizierungsprozess finden sich unter Abschnitt 6.2.6

- GET `api/auth/login`: Der Endpunkt leitet den Nutzer mit der entsprechend aufgerufenen URL zur Google-Anmeldeseite weiter.
- GET `api/auth/logout`: Der Nutzer wird abgemeldet.

- GET `api/auth/me`: Beim Laden der Frontendanzwendung werden hier die Daten wie Name und Profilbild geladen. Ist hier der Nutzer noch nicht in unserer Datenbank hinterlegt, wird er angelegt

### Group

- GET `api/group/`: Die Response gibt alle vorhandenen Lerngruppen zurück.
- GET `api/group/{groupId}`: Die Response gibt die Gruppe mit der entsprechenden ID zurück.
- POST (protected) `api/group`: Nimmt aus dem Request-Body ein GroupDTO-Objekt sowie den Token des Nutzers entgegen und erstellt daraufhin eine neue Gruppe.
- POST (protected) `api/group/join/{groupId}`: Fügt den eingeloggten Nutzer der Gruppe mit der entsprechenden groupId hinzu.
- POST `api/group/leave/{groupId}`: Entfernt den Nutzer aus der Gruppe mit der entsprechenden groupId.
- GET `api/group/{groupId}/calendar`: Response mit dem Gruppenkalender.

### User

- GET `api/user`: Response mit allen registrierten Nutzern.
- GET `api/user/{userId}`: Response des Nutzers mit der entsprechenden ID.
- PUT `api/user`: Nimmt aus dem Request-Body das UserDTO-Objekt des aktualisierten Nutzers entgegen.
- DELETE `api/user/{userId}`: Löscht den Nutzer aus der Datenbank.
- POST `api/user/raumzeit/login`: Nimmt aus dem Request-Body die RaumZeit-Credentials entgegen und ordnet die RaumZeit-Kalenderdaten dem Nutzerkonto zu.
- GET (protected) `api/user/calendar`: Die Response enthält die Kalenderdaten des Nutzers.

## 6.2.3. Kalenderdaten-Verarbeitung

Die Kernlogik zur Ermittlung gemeinsamer Termine ist im `GroupCalendarService` implementiert. Dieser führte folgende Schritte nacheinander aus:

### 1. Laden der Gruppe

- Die Gruppe wird anhand ihrer eindeutigen ID aus dem Repository geladen.
- Existiert die Gruppe nicht, wird kein Gruppenkalender erzeugt.

### 2. Abrufen der persönlichen Kalender

- Für jedes Gruppenmitglied wird der persönliche Kalender über den Kalenderservice geladen.
- Die geladenen Kalender werden zu einer gemeinsamen Sammlung zusammengeführt.

### 3. Zusammenführen aller Termine

- Alle Termine aus den persönlichen Kalendern werden in einer Liste gesammelt.
- Die Termine werden nach ihrer Startzeit sortiert.

### 4. Ermittlung der belegten Zeiträume

- Die sortierte Terminliste wird nacheinander durchlaufen.
- Für jeden Termin wird geprüft, ob er sich mit dem zuletzt gespeicherten Zeitraum überschneidet.
  - Keine Überschneidung: Es wird ein neuer Belegungsblock angelegt.

- Überschneidung: Die Zeiträume werden zu einem gemeinsamen Block zusammengeführt, indem das spätere Enddatum übernommen wird.
- Dadurch entstehen zusammenhängende Zeitintervalle, die alle belegten Zeiten der Gruppenmitglieder abdecken.

#### **6.2.4. Gruppenverwaltung**

Um eine Gruppe zu erstellen oder beizutreten, ist vorausgesetzt, dass alle Mitglieder zuvor ihren eigenen Stundenplan eingepflegt haben.

Gruppen können von jedem Nutzer erstellt werden. Dies geschieht auf der Hauptseite über den Button „*Gruppe erstellen*„. Im daraufhin erscheinenden Dialogfenster kann ein Gruppenname vergeben werden, nach dessen Bestätigung die Gruppe erstellt wird. Um weitere Nutzer zu einer Gruppe hinzuzufügen, müssen diese vom Ersteller eingeladen werden. Die Einladung kann über einen Link oder das Scannen eines QR-Codes erfolgen, wofür der Nutzer auf der GroupCard auf das QR-Icon tippt.

Der Gruppenkalender kann durch einen Klick auf die Gruppenkarte aufgerufen werden. Er stellt das Ergebnis aller einzelnen Kalender der Gruppenmitglieder dar. Im Gruppenkalender werden ausschließlich bereits belegte Zeitblöcke angezeigt. Durch einen Klick auf eine freie Spalte lässt sich ein Gruppen-Event hinzufügen.

Zum Verlassen einer Gruppe wird der Button „*Gruppe verlassen*„ oberhalb des Gruppenkalenders betätigt.

Die Verknüpfung zwischen Gruppen und ihren Mitgliedern ist in der Datenbank hinterlegt. Näheres dazu findet sich im folgenden Kapitel. Sind einer Gruppe keine Nutzer mehr zugeordnet, wird diese automatisch aus der Datenbank entfernt.

## 6.2.5. Datenbank

### 6.2.5.1. Datenbankstruktur

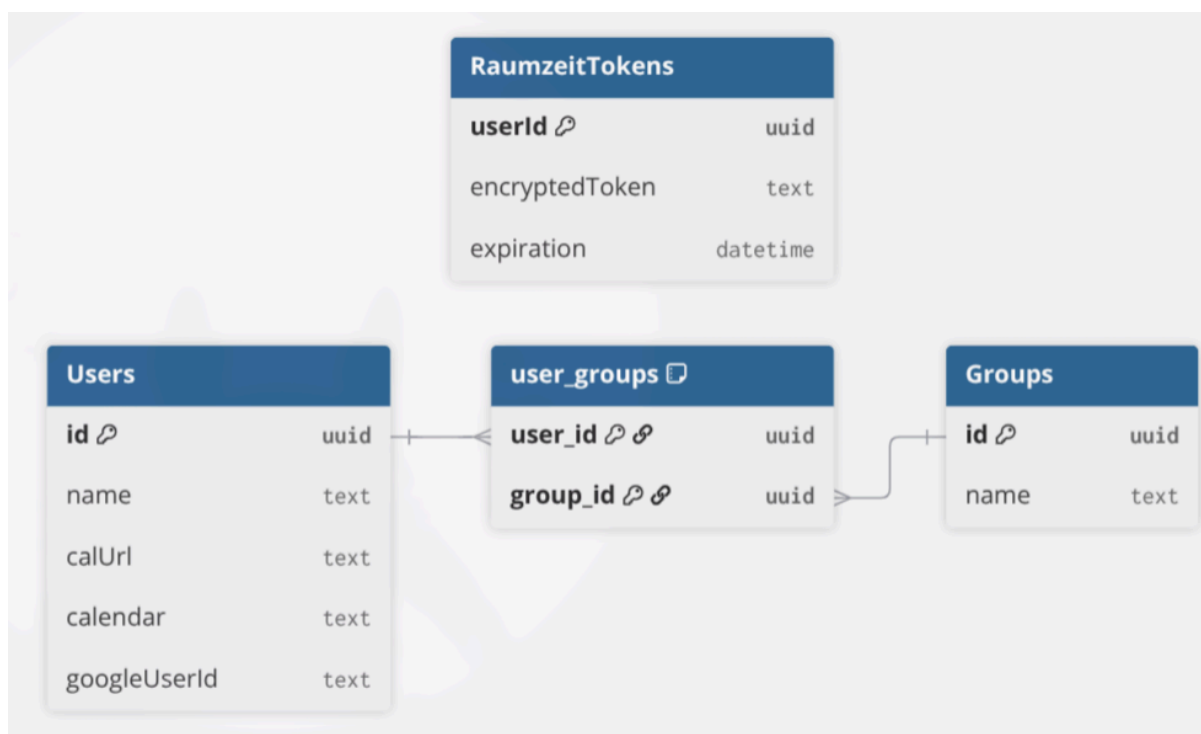


Abbildung 8: Datenbank-Schema

Abbildung 8 zeigt das Datenbankschema zwischen der User- und der Gruppentabelle. Die `user_groups`-Tabelle ist eine Relationstabelle und steht zu den Tabellen `users` und `groups` jeweils in einer 1-zu-n-Beziehung. Die Relation wird über die Primärschlüssel von `Users` und `Groups` hergestellt. Zur Authentifizierung bei der RaumZeit-API werden zusätzlich die dafür erforderlichen RaumZeit-Tokens der User verschlüsselt in der Datenbank abgelegt.

### 6.2.5.2. Implementierung

Das gewählte Datenbankmanagementsystem ist PostgreSQL. Im Rahmen der Domain-Driven-Architektur erfolgt der Datenbankzugriff der LernZeit-Anwendung über den `LernZeit.PostgresAdapter`. Dieser enthält unter anderem den Datenbankkontext, die für das .NET-Framework EntityFrameworkCore erforderlichen Migrationen sowie die Repositories zu den Gruppen- und Nutzerobjekten, Datenbank-Entity-Klassen und Mapper-Klassen.

Im `LernZeitDBContext` werden die Datenbank-Entities festgelegt und die `UserGroups`-Relation hergestellt. Das `Group`- und das `UserRepository` stellen diverse Schnittstellen zur Modifizierung der Datenbankobjekte bereit.

## 6.2.6. Authentifizierung

Die Anwendung nutzt das Backend-for-Frontend (BFF) Architekturmuster, um die Benutzerauthentifizierung abzusichern. Statt die sensiblen Anmeldedaten und Token direkt im Webbrowser (Frontend) zu verarbeiten, übernimmt der Server (Backend) die gesamte Kommunikation mit Google.

1. **Anmeldung über Google:** Wenn sich ein Nutzer anmeldet, wird er sicher zu Google weitergeleitet. Nach erfolgreichem Login schickt Google die Bestätigung und die Zugriffstoken direkt an das Backend, nicht an den Browser.
2. **Sicherer Datentresor (Server):** Der Server behält diese Token sicher bei sich. Der Browser bekommt den Token nie ausgestellt. Dadurch können sie nicht durch z. B. Cross-Site-Scripting abgegriffen werden.
3. **Abgesichertes Sitzungs-Cookie:** Als Ersatz für die Token stellt der Server einen eigenen HTTPS-ONLY Session-Cookie aus. Dieser Cookie ist für JavaScript im Browser unsichtbar und wird nur über verschlüsselte Verbindungen (HTTPS) transportiert. Der Browser sendet es bei jeder zukünftigen Anfrage automatisch im Hintergrund mit.
4. **API-optimierte Kommunikation:** Wenn ein nicht angemeldeter Nutzer versucht, auf geschützte Daten zuzugreifen, schickt der Server keine unschönen Webseiten-Weiterleitungen, sondern eine direkte, standardisierte Fehlermeldung (HTTP 401/403). Das Frontend kann diese Fehler abfangen und den Nutzer elegant zur Anmeldung führen.

Die Authentifizierung über Google erspart uns den Umgang von Zugangsdaten der Nutzer:innen. Nach erfolgreicher Anmeldung wird in den Endpunkten die `GoogleUserId` extrahiert. Über diese kann unsere interne User-Entität identifiziert werden.

#### 6.2.7. Kommunikation mit der RaumZeit API

Um den manuellen Aufwand für Studierende zu minimieren, integriert **Lernzeit** die RaumZeit-API der Hochschule.

- **RaumzeitService:** Diese Komponente kapselt die HTTP-Kommunikation mit dem Hochschul-System. Sie übernimmt das Mapping der proprietären RaumZeit-Strukturen auf unsere internen Domain-Modelle.
- **Sicherheit & Token-Management:** Da der Zugriff auf Stundenpläne autorisiert erfolgen muss, speichern wir die notwendigen Zugriffstoken verschlüsselt in der PostgreSQL-Datenbank. Hierfür kommt der `TokenEncryptionService` zum Einsatz, der die `IDataProtectionProvider`-Infrastruktur von .NET nutzt.

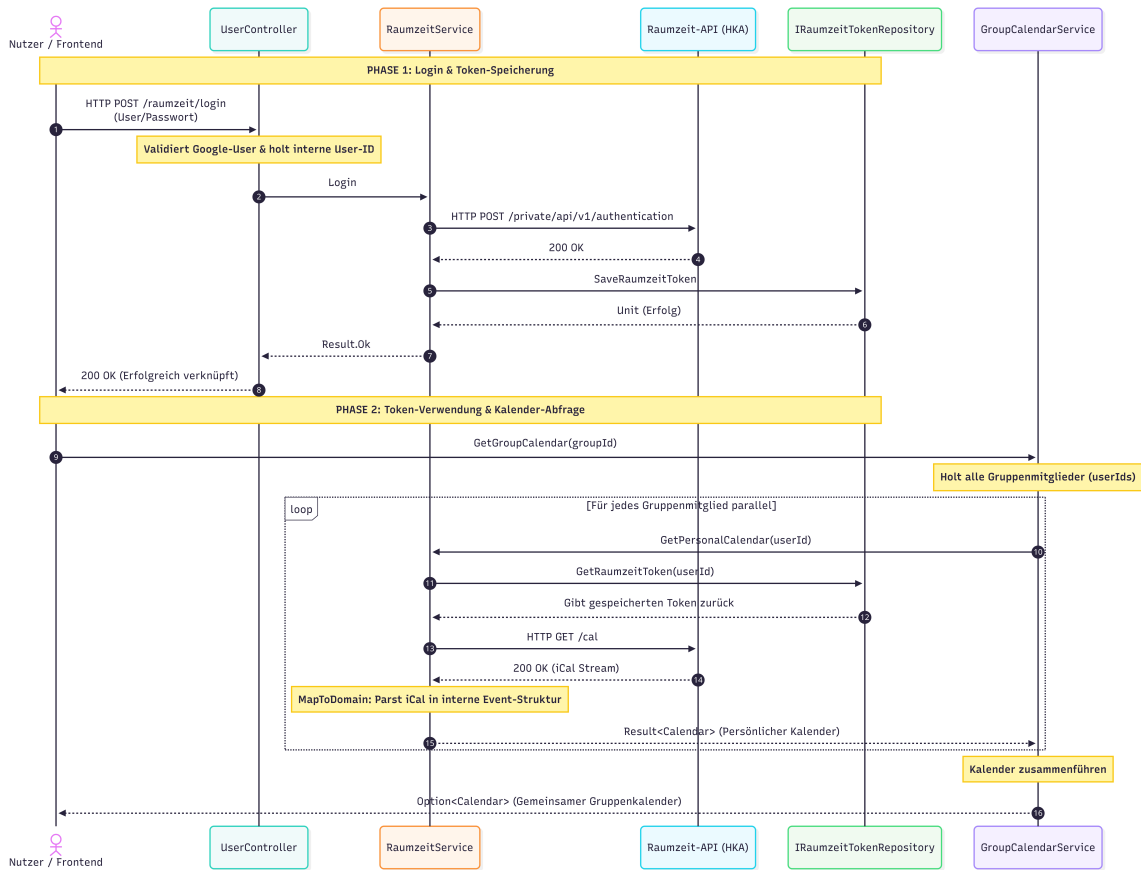


Abbildung 9: Raumzeit Tokenverwaltung & Nutzung

### 6.3. Testbarkeit

Die gewählte Architektur ermöglicht eine hohe Testbarkeit der Anwendung. Die Trennung von Domänenlogik und Adaptern erlaubt es, die Kernlogik isoliert zu testen, ohne dass externe Systeme (wie Datenbanken oder Drittanbieter-APIs) involviert sind.

Um die funktionale Korrektheit der API-Schnittstellen und des Datenflusses verlässlich zu gewährleisten, wurde eine automatisierte Integrations-Testsuite implementiert. Im Gegensatz zu klassischen Unit-Tests, die interne Services isoliert unter starkem Mocking-Einsatz prüfen, fokussieren sich diese Integrationstests auf die Überprüfung an der tatsächlichen Systemgrenze.

Das Bootstrapping des Backends erfolgt über die von Microsoft bereitgestellte WebApplicationFactory, welche im Speicher einen virtuellen TestHost hochfährt. Test-Requests werden über einen vom TestHost bereitgestellten HttpClient direkt an das Routing- und Middleware-System übergeben. Dadurch lassen sich Routing, Modell-Validierungen, DTO-Mapping, Authentifizierungs-Filter und HTTP-Antworten ohne echten Netzwerk-Overhead simulieren und testen.

Da In-Memory-Datenbanken (wie SQLite oder die EF Core In-Memory-Datenbank) PostgreSQL-spezifische SQL-Features, Trigger oder n:m-Kaskadierungen nicht originalgetreu abbilden, kommt für die Persistenzprüfung das Framework Testcontainers zum Einsatz. Bei der Initialisierung der Testklasse startet Testcontainers vollautomatisch einen temporären PostgreSQL-Docker-Container.

Dieser Ansatz bietet wesentliche Vorteile für die Softwarequalität:

1. **Realitätsnahe Validierung:** Es wird gegen ein echtes PostgreSQL-Datenbanksystem getestet, wodurch datenbankspezifische Eigenheiten (wie Foreign-Key-Constraints in der `user_groups`-Relationstabelle) exakt geprüft werden.
2. **Isolierte Sandbox-Umgebung:** Jeder Testlauf startet in einer sauberen Datenbankumgebung ohne verbleibende Seiteneffekte vorheriger Durchläufe.
3. **Zero-Configuration-Testing:** Entwickler und CI/CD-Runner (z. B. GitHub Actions) benötigen außer einer installierten Docker-Engine keine lokal konfigurierte PostgreSQL-Instanz.

## 6.4. Deployment und Infrastruktur

Das Projekt verfolgt einen modernen „Infrastructure as Code“ Ansatz.

### 6.4.1. Deployment-Umgebung

Die Applikation ist so konzipiert, dass sie vollständig in Docker-Containern lauffähig ist. Dies garantiert eine identische Laufzeitumgebung zwischen Entwicklung (Localhost), Testing und Produktion. Als Datenbank wird eine PostgreSQL-Instanz verwendet, die ebenfalls containerisiert bereitgestellt wird.

### 6.4.2. Containerisierung und Orchestrierung mit .NET Aspire

Ein technologisches Highlight ist der Einsatz von .NET Aspire. Dies ermöglicht eine nahtlose Orchestrierung der Microservices (oder modularen Monolithen) während der Entwicklung:

- **Service Discovery:** Das Frontend findet das Backend automatisch über logische Namen, statt über statische IP-Adressen.
- **Observability:** .NET Aspire bietet ein integriertes Dashboard, in dem Logs, Traces und Metriken aller Komponenten in Echtzeit eingesehen werden können.
- **Konfigurationsmanagement:** Connection Strings für die Datenbank und API-Keys werden zentral im AppHost verwaltet und sicher an die jeweiligen Services durchgereicht.

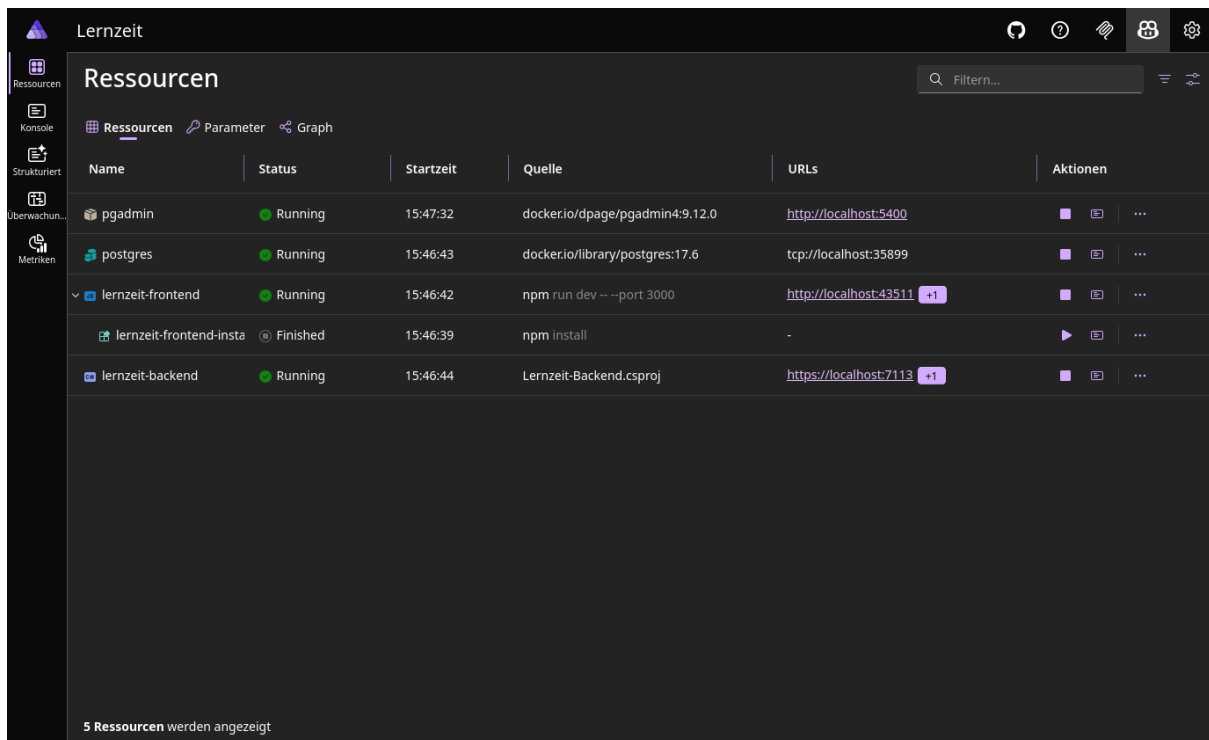


Abbildung 10: .NET Aspire Dashboard

## 7. Fazit

### 7.1. Rückblick

Im Rahmen dieses Moduls lag der Fokus auf der Software-Architektur des Projekts. Ein besonderes Augenmerk galt dabei der Strukturierung und Einordnung von Zuständigkeitsbereichen. Die Entscheidung fiel auf ein Domain-Driven-Modell, das eine klare Trennung zwischen Anwendungslogik und Adaptern gewährleistet.

Eine zentrale Fragestellung während der Ausarbeitung war es festzulegen, welche Nutzerdaten nötig sind, damit die Anwendung funktioniert. Das Ziel war es, nur das Nötigste in der Datenbank zu hinterlegen, da der Großteil der nutzerbezogenen Daten von Drittanbietern wie Google Authentication und der RaumZeit-API stammen.

Ebenfalls war die Authentifizierung der Nutzer zu Beginn ein wichtiges Thema. Die Auslagerung dieses Problems an einen Drittanbieter stellte sich als komfortable und zuverlässige Lösung heraus.

Im Frontend war zu Beginn die Auswahl einer geeigneten Kalenderkomponente eine Herausforderung. Die Entscheidung für eine vorgefertigte React-Kalenderkomponente ermöglichte ein schnelleres Ergebnis, anstatt alle Funktionen von Grund auf selbst zu entwickeln. Da die Auswahl an verfügbaren Komponenten groß ist und jede Komponente anders funktioniert, wurden mithilfe des KI-Modells *BigPickle* von OpenCode mehrere Komponenten iterativ getestet, um die für den Anwendungsfall am besten geeignete zu ermitteln.

Im Verlauf des Entwicklungsprozesses kam es vereinzelt dazu, dass Komponenten fehlerhafte Zustände annahmen oder nicht mehr korrekt funktionierten, was eine weitere Herausforderung darstellte.

Darüber hinaus funktionierte die Authentifizierung aus verschiedenen Gründen nicht immer zuverlässig. Insbesondere die CORS-Policies stellten zu Beginn aufgrund von Client-Weiterleitungen ein Problem dar.

Gegen Ende des Projekts ergaben sich in der Testphase weitere Herausforderungen. Das Testen der Anwendung mit mehreren Nutzern ist nicht trivial, da Nutzerkonten eng mit dem jeweiligen Google-Account verknüpft sind. Daher waren zusätzliche Google-Accounts erforderlich, um die Anwendung vollständig testen zu können.

## **7.2. Ausblick**

Das LernZeit-Projekt hat alle Mindestanforderungen erfüllt, die zu Beginn im Pflichtenheft festgelegt wurden. Es ist möglich, Lerngruppen zu erstellen, eigene Kalender aus RaumZeit zu importieren, diese mit den Mitgliedern der Lerngruppe abzugleichen und in einer übersichtlichen Benutzeroberfläche einen gemeinsamen freien Zeitraum festzulegen.

Neben den funktionalen Anforderungen sind auch die nicht-funktionalen Anforderungen erfüllt. Die LernZeit-Anwendung entspricht den Prinzipien des Domain-Driven Designs: Komponenten sind klar in ihren Zuständigkeitsbereichen getrennt, was eine reibungslose Wartung ermöglicht. Ebenso können Komponenten bei Bedarf unkompliziert ausgetauscht oder erweitert werden. Darüber hinaus bietet eine klare Trennung eine einfache Möglichkeit Komponenten isoliert zu testen.

Für zukünftige Erweiterungen wären weitere Use-Cases denkbar. So wäre eine Funktion zur gemeinsamen Terminabstimmung innerhalb einer Gruppe eine sinnvolle Ergänzung. Darüber hinaus würde eine Anzeige aller freien Räume auf dem Campus die Suche nach einem geeigneten Lernort erleichtern. Die Integration von Benachrichtigungen bei neuen Terminvorschlägen oder Änderungen wäre eine nützliche Erinnerungsfunktion. Schließlich wäre auch die Anbindung weiterer Kalender-Apps, beispielsweise Google Calendar oder Outlook, eine denkbare Erweiterung.

## Literaturverzeichnis

- [1] Zugriffen: 15. Juni 2025. [Online]. Verfügbar unter: <https://www.figma.com/resource-library/ai-in-design/>
- [2] Zugriffen: 22. Juni 2025. [Online]. Verfügbar unter: <https://in.pinterest.com/pin/53339576832700058/>
- [3] Zugriffen: 22. Juni 2025. [Online]. Verfügbar unter: <https://se.pinterest.com/pin/20969954512913816/>
- [4] Zugriffen: 22. Juni 2025. [Online]. Verfügbar unter: <https://developers.google.com/identity/siwg>

## Hinweise auf KI-Unterstützung

Zur Umsetzung des Projekts wurde auf verschiedene KI-gestützte Tools zurückgegriffen. Diese wurden insbesondere für die Erstellung von Prototypen, die Generierung von Code-Snippets und die Unterstützung bei der Implementierung genutzt. Die folgenden Tools kamen zum Einsatz:

- Figma AI: Für die Erstellung von UI-Prototypen und Designideen
- OpenCode + BigPickle: Für die Auswahl und Evaluierung von React-Komponenten
- OpenCode + Gemini 3 (Flash und Pro):
  - Erster Entwurf für .NET Aspire Anbindung (Wurde größtenteils wieder verworfen, da die Anbindung über die .NET Aspire Dokumentation doch besser nachvollziehbar war)
  - Vervollständigung von Testsetup: Hierbei wurde folgender Prompt genutzt: *„I started a integration test project for lernzeit-backend. Finish setting up Testcontainers for the postgres db and write tests covering the group and user controller“*. Die Anschließend generierten Tests wurden in Teilen übernommen und erweitert um die gewünschten Funktionalitäten abzudecken.
  - Die Button Komponente des Frontends wurde mit Hilfe von Gemini erstellt um zu lernen wie Komponenten in React funktionieren. Hierzu wurde folgender Prompt genutzt: *„create a react button component with different states like primary secondary ghost“*. Zusätzlich wurde ein Screenshot aus dem Figma Design hinzugefügt. Die entstandene Komponente wurde dann um Funktionen wie das Anzeigen von Icons erweitert. Zudem wurden die exakten Designvorgaben aus Figma übernommen, da die generierte Komponente nicht exakt den Designvorgaben entsprach.